

The first command ensures that you have the latest version of the pip program. The second line installs the most recent version of pygame.

If you have any problems installing pygame, consult the pygame documentation at <https://www.pygame.org/wiki/GettingStarted>. To test that pygame has been installed correctly, open IDLE (the development environment that is bundled with the default implementation of Python), and in the shell window enter:

```
import pygame
```

If you see a message saying something like “Hello from the pygame community” or if you get no message at all, then pygame has been installed correctly. The lack of an error message indicates that Python has been able to find and load the pygame package and it’s ready to use. If you would like to see a sample game using pygame, enter the following command (which starts a version of *Space Invaders*):

```
python3 -m pygame.examples.aliens
```

Before we get into using pygame, I need to explain two important concepts. First, I’ll explain how individual pixels are addressed in programs that use a GUI. Then, I’ll discuss event-driven programs and how they differ from typical text-based programs. After that, we’ll code a few programs that demonstrate key pygame features.

Window Details

A computer screen is made up of a large number of rows and columns of small dots called *pixels* (from the words *picture element*). A user interacts with a GUI program through one or more windows; each window is a rectangular portion of the screen. Programs can control the color of any individual pixel in their window(s). If you’re running multiple GUI programs, each program is typically displayed in its own window. In this section, I’ll discuss how you address and alter individual pixels in a window. These concepts are independent of Python; they are common to all computers and are used in all programming languages.